École Marocaine des Sciences de l'Ingénieur (EMSI)

Projet : Analyse de Sentiments

Travaux Pratiques 2 & 3

Étudiantes : Hafsa Boutayeb & Aya Soulaimane

Professeur : Pr. Abderrahim Larhlimi

Date : 14 octobre 2026

Année Universitaire 2025 - 2026

Résumé

Ce document présente le compte-rendu détaillé des Travaux Pratiques 2 et 3 réalisés dans le cadre du cours d'Intelligence Artificielle à l'EMSI. L'objectif principal de ce projet était de concevoir et déployer un système complet d'analyse de sentiments capable de classifier automatiquement des avis clients en catégories positives ou négatives.

Le TP2 s'est concentré sur l'entraînement et l'évaluation d'un modèle de classification de texte en utilisant l'approche "sac de mots" (TF-IDF) combinée à un classifieur Ridge. Nous avons travaillé sur deux jeux de données : un jeu de démonstration de 290 avis et un jeu plus conséquent de 20 000 avis provenant de la plateforme Allociné. Les résultats obtenus montrent un score F1 macro de 0.879 sur le jeu de démonstration et de 0.920 sur le jeu Allociné, dépassant ainsi le seuil de performance requis de 0.85.

Le TP3 a consisté à développer une interface utilisateur interactive et professionnelle en utilisant le framework Streamlit. Cette interface permet aux utilisateurs d'analyser des avis individuels ou des fichiers CSV complets, avec affichage des prédictions, des niveaux de confiance et de statistiques détaillées. L'application intègre également un système de seuil d'incertitude (60%) permettant d'identifier les avis ambigus nécessitant une vérification humaine.

Mots-clés : Analyse de sentiments, Machine Learning, TF-IDF, Streamlit, Classification de texte, Python, Intelligence Artificielle.

Remerciements

Nous tenons à exprimer notre profonde gratitude à notre professeur, Pr. Abderrahim Larhlimi, pour son encadrement précieux, ses conseils avisés et sa disponibilité tout au long de ce projet. Ses explications claires et sa pédagogie nous ont permis de comprendre les concepts fondamentaux de l'analyse de sentiments et du machine learning.

Nous remercions également l'École Marocaine des Sciences de l'Ingénieur (EMSI) pour nous avoir fourni les ressources et l'environnement d'apprentissage nécessaires à la réalisation de ce travail.

Enfin, nous nous remercions mutuellement pour notre collaboration fructueuse et notre engagement dans la réussite de ce projet.

Table des matières

Résumé **1**

Remerciements **2**

Introduction Générale **5**

1 TP2 : Entraînement et Évaluation du Modèle de Sentiment **6**
1.1 Présentation du TP2 6
1.1.1 Objectifs Pédagogiques 6
1.1.2 Architecture du Projet 6
1.2 Préparation des Données 6
1.2.1 Génération du Jeu de Démonstration 6
1.2.2 Vectorisation TF-IDF 7
1.3 Entraînement du Modèle 7
1.3.1 Méthodologie 7
1.3.2 Résultats sur le Jeu de Démonstration 7
1.3.3 Résultats sur le Jeu Allociné 8
1.4 Évaluation et Analyse des Erreurs 8
1.4.1 Métriques de Performance 8
1.4.2 Matrice de Confusion 8
1.4.3 Analyse Qualitative des Erreurs 9
1.4.4 Explication des Erreurs 9
1.4.5 Pistes d'Amélioration 9
1.5 Validation des Tests 10
1.5.1 Suite de Tests 10

2 TP3 : Interface de Démonstration avec Streamlit **11**
2.1 Présentation du TP3 11
2.1.1 Objectifs 11
2.1.2 Choix Technologique : Streamlit 11
2.2 Architecture de l'Application 11
2.2.1 Structure des Fichiers 11
2.2.2 Fonctionnement 12
2.3 Fonctionnalités Détaillées 12
2.3.1 Onglet 1 : Analyse d'un Avis Individuel (Contrôle S1 & S2) 12
2.3.2 Onglet 2 : Analyse d'un Fichier CSV (Contrôle S3) 13
2.3.3 Onglet 3 : Documentation "Comment ça marche" 13
2.4 Tests et Validation (Contrôle S4) 14
2.4.1 Suite de Tests Unitaires 14

2.4.2 Résultats 14
2.5 Démonstration et Captures d'Écran 14
2.5.1 Lancement de l'Application 14
2.5.2 Interface Utilisateur 14

3 Conclusion Générale et Perspectives 15
3.1 Bilan du Projet 15
3.1.1 Objectifs Atteints 15
3.1.2 Compétences Acquises 15
3.2 Limites Identifiées 16
3.2.1 Limites du Modèle "Sac de Mots" 16
3.2.2 Limites de l'Interface 16
3.3 Perspectives d'Amélioration 16
3.3.1 Améliorations du Modèle 16
3.3.2 Améliorations de l'Interface 17
3.3.3 Déploiement en Production 17
3.4 Apport Personnel 17
3.5 Mot de la Fin 17

Introduction Générale

Contexte du Projet

Dans un monde où les avis clients en ligne jouent un rôle crucial dans les décisions d'achat et la réputation des entreprises, l'analyse automatique de sentiments est devenue un outil indispensable. Ce projet s'inscrit dans cette problématique en proposant une solution complète allant de l'entraînement d'un modèle de classification à son déploiement via une interface utilisateur conviviale.

Objectifs

Les objectifs de ce projet sont multiples :

— Maîtriser les techniques de prétraitement de texte et de vectorisation (TF-IDF)

— Entraîner et évaluer un modèle de classification supervisée

— Comprendre les limites des approches "sac de mots" face aux avis complexes

— Développer une interface de démonstration professionnelle et interactive

— Mettre en place une démarche de développement rigoureuse avec tests et versioning

Structure du Document

Ce rapport est structuré en trois chapitres principaux. Le premier chapitre détaille la méthodologie et les résultats du TP2 (entraînement et évaluation du modèle). Le second chapitre présente le développement de l'interface Streamlit (TP3). Enfin, le troisième chapitre propose une conclusion générale et des perspectives d'amélioration.

Chapitre 1

TP2 : Entraînement et Évaluation du Modèle de Sentiment

1.1 Présentation du TP2

1.1.1 Objectifs Pédagogiques

Le TP2 avait pour objectif principal de nous familiariser avec les étapes fondamentales du machine learning appliqué au traitement du langage naturel (NLP). Plus précisément, il s'agissait de :

— Préparer et vectoriser des données textuelles

— Entraîner un classifieur supervisé

— Évaluer les performances du modèle à l'aide de métriques appropriées

— Analyser les erreurs de prédiction pour comprendre les limites du modèle

1.1.2 Architecture du Projet

Le projet suit une architecture modulaire organisée comme suit :

— **src**/ : Contient les scripts Python principaux (make_demo.py, train.py, evaluate.py, predict.py)

— **tests**/ : Répertoire des tests unitaires pour valider le code

— **models**/ : Stockage des modèles entraînés et des métriques

— **data**/ : Jeux de données d'entraînement et de test

— **notebooks**/ : Notebooks Jupyter pour l'exploration

1.2 Préparation des Données

1.2.1 Génération du Jeu de Démonstration

La première étape a consisté à générer un jeu de données de démonstration en exécutant le script `python src/make_demo.py`. Ce script a produit un fichier `data/demo_reviews.csv` contenant 290 avis répartis équitablement entre positifs et négatifs (145 de chaque catégorie).

La particularité de ce jeu de données réside dans l'inclusion volontaire d'avis "mitigés" (100 avis) et "neutres" (30 avis). Ces avis contiennent des retournements de situation, typiquement introduits par des expressions comme "sauf que ensuite", "mais finalement", ou "au début... mais". Par exemple :

Exemple d'avis mitigé

"Service impeccable au début, mais finalement produit cassé à la réception."

Ces cas complexes sont intentionnellement inclus pour tester la robustesse du modèle face aux nuances du langage naturel.

1.2.2 Vectorisation TF-IDF

Le prétraitement des données textuelles a été réalisé à l'aide de la méthode TF-IDF (Term Frequency - Inverse Document Frequency). Cette approche permet de :

— Convertir le texte en représentations numériques

— Pondérer l'importance des termes en fonction de leur fréquence dans le document et dans le corpus global

— Réduire l'impact des mots très fréquents mais peu informatifs

Pour le jeu de démonstration, le vocabulaire extrait comportait 316 termes uniques, tandis que pour le jeu Allociné complet, le vocabulaire s'élevait à 149 529 termes.

1.3 Entraînement du Modèle

1.3.1 Méthodologie

Nous avons utilisé un classifieur Ridge (régression linéaire régularisée) combiné à la vectorisation TF-IDF. Cette approche, bien que simple, offre de bonnes performances pour la classification de texte et reste interprétable.

Le processus d'entraînement suit les étapes suivantes :

1. Chargement des données depuis un fichier CSV ou Parquet
2. Séparation en ensemble d'entraînement (80%) et de test (20%)
3. Vectorisation TF-IDF des textes
4. Entraînement du classifieur Ridge sur les données d'entraînement
5. Évaluation sur les données de test
6. Sauvegarde du modèle et des métriques

1.3.2 Résultats sur le Jeu de Démonstration

L'exécution de la commande `python src/train.py` a produit les résultats suivants :

Résultats - Jeu de Démo

Source : demo, 290 avis (145 négatifs)
Répartition : 232 avis d'entraînement, 58 de test
Vocabulaire : 316 termes
F1 macro sur le jeu de test : 0.879 (seuil requis : 0.85)

Le score F1 macro de 0.879 dépasse le seuil exigé de 0.85, validant ainsi le Contrôle C3.

1.3.3 Résultats sur le Jeu Allociné

Pour évaluer la robustesse du modèle sur un jeu de données plus conséquent, nous avons exécuté `python src/train.py -allocine`. Cette commande a téléchargé 20 000 avis de films (7,6 Mo) depuis une source distante et a produit les résultats suivants :

Résultats - Jeu Allociné

Source : allocine, 20 000 avis (10 204 négatifs)
Répartition : 16 000 avis d'entraînement, 4 000 de test
Vocabulaire : 149 529 termes
F1 macro sur le jeu de test : 0.920 (seuil requis : 0.85)

L'augmentation significative du score F1 (de 0.879 à 0.920) s'explique par la plus grande quantité de données d'entraînement, permettant au modèle de mieux généraliser. Ce résultat valide le Contrôle C5.

1.4 Évaluation et Analyse des Erreurs

1.4.1 Métriques de Performance

L'exécution de `python src/evaluate.py` a généré un rapport de classification détaillé :

TABLE 1.1 – Métriques de classification sur le jeu de test (58 avis)

Classe	Précision	Rappel	F1-score	Support
Négative	0.844	0.931	0.885	29
Positive	0.923	0.828	0.873	29
Accuracy			0.879	58
Macro avg	0.883	0.879	0.879	58

Ces métriques montrent un bon équilibre entre les deux classes, avec une légère préférence pour la détection des avis négatifs (rappel de 93.1%).

1.4.2 Matrice de Confusion

L'analyse de la matrice de confusion révèle :

— **27 vrais négatifs** correctement classés
— **24 vrais positifs** correctement classés
— **2 faux positifs** (négatifs classés positifs)
— **5 faux négatifs** (positifs classés négatifs)

Soit un total de **7 erreurs sur 58 avis**, ce qui représente un taux d'erreur de 12%.

1.4.3 Analyse Qualitative des Erreurs

L'examen des exemples d'erreurs révèle un pattern clair : tous les avis mal classés sont des avis mitigés contenant des retournements de situation. Voici trois exemples significatifs :

Erreurs de Classification

Exemple 1 :
Vérité : positive | Prédit : negative
"Taille trop petite et retour payant au déballage, sauf que ensuite produit conforme à la description."

Exemple 2 :
Vérité : positive | Prédit : negative
"Commande jamais reçue et remboursement en attente au début, sauf que ensuite je recommande."

Exemple 3 :
Vérité : negative | Prédit : positive
"Service impeccable et livraison en avance sur le papier, sauf que ensuite produit cassé à l'arrivée."

1.4.4 Explication des Erreurs

Ces erreurs s'expliquent par la nature même de l'approche "sac de mots" (bag-of-words) utilisée par TF-IDF. Cette méthode :

— Compte les occurrences de mots positifs et négatifs sans tenir compte de leur ordre
— Ne capture pas la sémantique des connecteurs logiques ("mais", "sauf que", "cependant")
— Ignore la structure temporelle et les retournements de situation

Dans l'exemple *"Service impeccable... sauf que ensuite produit cassé"*, le modèle détecte à la fois des mots positifs ("impeccable", "avance") et négatifs ("cassé"), mais ne comprend pas que le mot "sauf que" inverse la polarité globale de l'avis.

1.4.5 Pistes d'Amélioration

Pour corriger ces limitations, plusieurs approches pourraient être envisagées :

— **Word Embeddings (Word2Vec, GloVe) :** Représentations vectorielles des mots capturant leur sens sémantique

— **Modèles de séquence (RNN, LSTM) :** Prise en compte de l'ordre des mots

— **Transformers (BERT, CamemBERT) :** Modèles contextuels pré-entraînés comprenant la syntaxe et la sémantique

— **Analyse de dépendances :** Identification des relations grammaticales entre les mots

Ces approches seront probablement abordées dans les semaines suivantes du cours.

1.5 Validation des Tests

1.5.1 Suite de Tests

L'exécution de `python -m pytest -q` après l'entraînement sur Allociné a validé l'ensemble des fonctionnalités :

Résultats des Tests

42 tests passés en 2.49 secondes

Ces tests couvrent :

— La génération des données de démonstration

— L'entraînement du modèle et la sauvegarde des métriques

— Les prédictions sur de nouveaux textes

— La cohérence des scores F1 avec les seuils requis

Chapitre 2

TP3 : Interface de Démonstration avec Streamlit

2.1 Présentation du TP3

2.1.1 Objectifs

Le TP3 avait pour objectif de rendre le modèle entraîné accessible aux utilisateurs non-techniques via une interface web interactive. Les objectifs spécifiques étaient :

— Développer une interface utilisateur conviviale avec Streamlit
— Permettre l'analyse d'avis individuels en temps réel
— Offrir la possibilité d'analyser des fichiers CSV complets
— Afficher les prédictions avec un niveau de confiance
— Implémenter un système de seuil d'incertitude
— Générer des visualisations statistiques

2.1.2 Choix Technologique : Streamlit

Streamlit a été choisi pour plusieurs raisons :

— **Simplicité :** Développement rapide en Python pur sans connaissances en HTML/CSS
— **Interactivité :** Composants riches (boutons, zones de texte, upload de fichiers)
— **Visualisation :** Intégration native de graphiques et tableaux
— **Déploiement :** Facilité de mise en production

2.2 Architecture de l'Application

2.2.1 Structure des Fichiers

L'interface se compose principalement de deux fichiers :

— **ui/app.py :** Application principale Streamlit contenant la logique d'interface
— **ui/helpers.py :** Fonctions utilitaires pour le chargement du modèle et les prédictions

2.2.2 Fonctionnement

L'application suit le flux suivant :

1. Chargement du modèle entraîné (`models/sentiment.joblib`) au démarrage
2. Affichage de trois onglets : "Un avis", "Un fichier CSV", "Comment ça marche"
3. Selon l'onglet sélectionné, l'utilisateur peut :
   — Saisir un texte et obtenir une prédiction immédiate
   — Uploader un fichier CSV pour analyse en batch
   — Consulter la documentation sur le fonctionnement

2.3 Fonctionnalités Détaillées

2.3.1 Onglet 1 : Analyse d'un Avis Individuel (Contrôle S1 & S2)

Interface

L'onglet "Un avis" présente :

— Une zone de texte multiligne pour la saisie de l'avis
— Un bouton "Analyser" pour déclencher la prédiction
— Un affichage dynamique du résultat avec :
   — Le verdict (positif/négatif/à vérifier)
   — Le niveau de confiance en pourcentage
   — Une barre de progression visuelle

Logique de Décision

Le système implémente un seuil d'incertitude de 60% :

— **Confiance $\geq$ 60% :** Verdict tranché ("positif" ou "négatif")
— **Confiance $<$ 60% :** Verdict "À vérifier" (nécessite intervention humaine)

Ce mécanisme permet de signaler les cas ambigus où le modèle hésite, conformément aux bonnes pratiques de déploiement de modèles de machine learning en production.

Exemple de Test

Lors du test avec l'avis : *"Service impeccable au début, mais finalement produit cassé à la réception."*

Résultat obtenu :

— **Prédiction :** Avis négatif : à traiter en priorité
— **Confiance :** 61%
— **Verdict :** Négatif (juste au-dessus du seuil)

Cet exemple illustre la capacité du modèle à détecter la polarité globale malgré la présence de termes positifs initiaux.

2.3.2 Onglet 2 : Analyse d'un Fichier CSV (Contrôle S3)

Interface

L'onglet "Un fichier CSV" permet :

- Le drag-and-drop ou la sélection d'un fichier CSV
- La spécification de la colonne contenant les avis (par défaut : "review")
- L'affichage des résultats sous forme de :
  - Statistiques globales (nombre d'avis, pourcentage de négatifs, nombre d'avis à vérifier)
  - Graphique en barres montrant la répartition des prédictions
  - Tableau détaillé avec colonnes : review, label, prediction, confiance, verdict
  - Bouton de téléchargement des résultats au format CSV

Test avec sample_reviews.csv

L'analyse du fichier `data/sample_reviews.csv` (30 avis) a produit les résultats suivants :

Résultats de l'Analyse CSV

Avis analysés : 30
Négatifs détectés : 20 (66.7%)
À vérifier : 7 avis (confiance $< 60\%$)

Le graphique en barres montre clairement la prédominance des avis négatifs dans cet échantillon, tandis que le tableau détaillé permet d'inspecter chaque prédiction individuellement.

Visualisations

Les visualisations générées incluent :

- **Bar chart :** Répartition des prédictions (negative vs positive)
- **Dataframe interactif :** Tableau triable et consultable
- **Indicateurs clés :** KPIs en haut de page pour une vue d'ensemble rapide

2.3.3 Onglet 3 : Documentation "Comment ça marche"

Cet onglet fournit une explication pédagogique du fonctionnement de l'application :

- **Ce que fait l'interface :** Elle charge le modèle produit au TP2 et appelle sa fonction `predict()`. Aucune logique de prédiction n'est dupliquée.
- **Le seuil d'incertitude :** En dessous de 60% de confiance, la prédiction est affichée comme "à vérifier" plutôt que tranchée. Un modèle qui hésite doit le dire : c'est ce qui permet à un agent de reprendre la main.
- **Ce que l'interface n'est pas :** Une démonstration, pas l'application finale. Elle sert à montrer le modèle, à récolter des retours et à faire apparaître les cas où il se trompe.

2.4 Tests et Validation (Contrôle S4)

2.4.1 Suite de Tests Unitaires

Le fichier `tests/test_ui.py` contient des tests pour valider :

— Le chargement correct du modèle

— La cohérence des prédictions entre l'interface et les scripts

— Le respect du seuil d'incertitude

— La gestion des fichiers CSV

2.4.2 Résultats

L'exécution de `python -m pytest -q` a produit :

Résultats des Tests TP3

45 tests passés en 2.94 secondes

Ce résultat porte le nombre total de tests du projet à 45 (incluant les 42 tests du TP2), tous validés avec succès.

2.5 Démonstration et Captures d'Écran

2.5.1 Lancement de l'Application

Pour lancer l'interface, la commande utilisée est :

```
streamlit run ui/app.py
```

Cette commande démarre un serveur web local accessible à l'adresse `http://localhost:8501`.

2.5.2 Interface Utilisateur

Les captures d'écran réalisées pendant les tests montrent :

— Une interface sombre et professionnelle

— Une navigation fluide entre les onglets

— Des visualisations claires et lisibles

— Une réactivité satisfaisante

Chapitre 3

Conclusion Générale et Perspectives

3.1 Bilan du Projet

3.1.1 Objectifs Atteints

Ce projet nous a permis de mener à bien un projet complet de machine learning, de la conception au déploiement :

- **TP2 - Modèle de Classification :**
  - Entraînement d'un modèle TF-IDF + Ridge
  - Score F1 macro de 0.920 sur 20 000 avis (seuil requis : 0.85)
  - Analyse approfondie des erreurs et compréhension des limites
  - 42 tests unitaires validés
- **TP3 - Interface Streamlit :**
  - Développement d'une interface web interactive
  - Analyse en temps réel d'avis individuels
  - Traitement par lots de fichiers CSV
  - Implémentation d'un seuil d'incertitude (60%)
  - 45 tests unitaires validés

3.1.2 Compétences Acquises

Au cours de ce projet, nous avons développé les compétences suivantes :

- **Techniques :**
  - Prétraitement et vectorisation de texte (TF-IDF)
  - Entraînement et évaluation de modèles de classification
  - Interprétation de métriques (précision, rappel, F1-score)
  - Développement d'applications web avec Streamlit
  - Écriture de tests unitaires avec pytest
  - Versioning de code avec Git et GitHub
- **Méthodologiques :**
  - Approche itérative de développement
  - Analyse critique des résultats
  - Documentation et compte-rendu technique
  - Travail en équipe et collaboration

3.2 Limites Identifiées

3.2.1 Limites du Modèle "Sac de Mots"

Comme l'ont montré les analyses d'erreurs, l'approche TF-IDF présente des limitations face aux :

- Avis mitigés avec retournements de situation
- Négations complexes ("pas mauvais", "rien d'exceptionnel")
- Ironie et sarcasme
- Contexte et dépendances à longue distance

3.2.2 Limites de l'Interface

L'interface actuelle, bien que fonctionnelle, pourrait être améliorée :

- Absence d'authentification et de gestion des utilisateurs
- Pas de persistance des analyses effectuées
- Visualisations basiques (pourraient être enrichies)
- Pas de système de feedback pour améliorer le modèle

3.3 Perspectives d'Amélioration

3.3.1 Améliorations du Modèle

Plusieurs pistes pourraient être explorées pour améliorer les performances :

1. **Word Embeddings (Semaine 9 du cours) :**
   - Utilisation de Word2Vec ou GloVe pour capturer la sémantique des mots
   - Similarité cosinus entre mots pour gérer les synonymes
2. **Modèles de Séquence :**
   - RNN (Recurrent Neural Networks) pour tenir compte de l'ordre des mots
   - LSTM (Long Short-Term Memory) pour capturer les dépendances à long terme
3. **Transformers (État de l'art) :**
   - Fine-tuning de BERT ou CamemBERT (pour le français)
   - Attention mechanism pour pondérer l'importance des mots
   - Scores F1 attendus > 0.95
4. **Data Augmentation :**
   - Génération d'avis synthétiques
   - Traduction automatique pour enrichir le corpus

3.3.2 Améliorations de l'Interface

1. **Fonctionnalités avancées :**
   - Authentification des utilisateurs
   - Historique des analyses
   - Export des résultats en PDF
   - Dashboard de suivi des performances
2. **Visualisations enrichies :**
   - Word clouds pour visualiser les termes importants
   - Graphiques temporels si données datées
   - Analyse des émotions (joie, colère, tristesse, etc.)
3. **Système de feedback :**
   - Boutons "Utile / Pas utile" pour chaque prédiction
   - Collecte des corrections pour réentraînement
   - Active learning pour identifier les cas les plus informatifs

3.3.3 Déploiement en Production

Pour une mise en production réelle, il faudrait :

- Déployer l'application sur Streamlit Cloud ou Heroku
- Mettre en place une API REST pour intégration avec d'autres systèmes
- Configurer un pipeline CI/CD pour les mises à jour automatiques
- Implémenter un système de monitoring des performances
- Prévoir un plan de scaling pour gérer la charge

3.4 Apport Personnel

Ce projet a été pour nous une expérience enrichissante à plusieurs niveaux :

- **Sur le plan technique :** Nous avons découvert les subtilités du NLP et les défis de la classification de texte. La confrontation entre les résultats théoriques et les erreurs réelles nous a appris l'importance de l'analyse critique.
- **Sur le plan méthodologique :** Nous avons compris l'importance des tests unitaires, du versioning Git et de la documentation. Ces pratiques, parfois perçues comme contraignantes, se révèlent essentielles pour un développement robuste.
- **Sur le plan humain :** Le travail en binôme nous a appris à collaborer efficacement, à partager les tâches et à conjuguer nos compétences pour atteindre un objectif commun.

3.5 Mot de la Fin

Ce projet d'analyse de sentiments nous a permis de toucher du doigt la complexité du traitement du langage naturel. Si les résultats obtenus sont satisfaisants (F1 = 0.920), ils nous ont surtout appris l'humilité face aux nuances du langage humain.

Comme l'ont montré les erreurs sur les avis mitigés, un modèle "sac de mots" reste limité face à la richesse syntaxique et sémantique du langage naturel. C'est pourquoi nous sommes impatientes d'aborder les semaines suivantes du cours, qui traiteront des embeddings de mots et des transformers, promettant des performances encore supérieures.

Nous tenons à réitérer nos remerciements au Pr. Abderrahim Larhlimi pour son encadrement de qualité et sa pédagogie, qui ont rendu ce projet à la fois stimulant et formateur.

Hafsa Boutayeb & Aya Soulaimane

14 octobre 2026